

Uso básico de Valgrind

Valgrind permite encontrar errores de memoria que muchas veces no son evidentes al ejecutar un programa normalmente. Entre otras cosas, puede detectar accesos inválidos, uso de valores no inicializados y memoria reservada que nunca fue liberada.

Esta guía muestra una forma simple de ejecutarlo y explica cómo leer las partes más importantes de su salida.

1 Antes de ejecutar Valgrind

Compila tu programa con información de depuración usando la opción `-g`. También conviene habilitar las advertencias del compilador:

```
gcc -Wall -Wextra -Wpedantic -g -o programa programa.c
```

El nos archivos `Makefile` que les daremos en el curso ya estarán configuradas todas estas flags, por lo que basta con compilar su código con `make`.

Antes de usar Valgrind, comprueba que el programa pueda ejecutarse normalmente:

```
./programa
```

2 Comando recomendado

Para una revisión general, puedes usar:

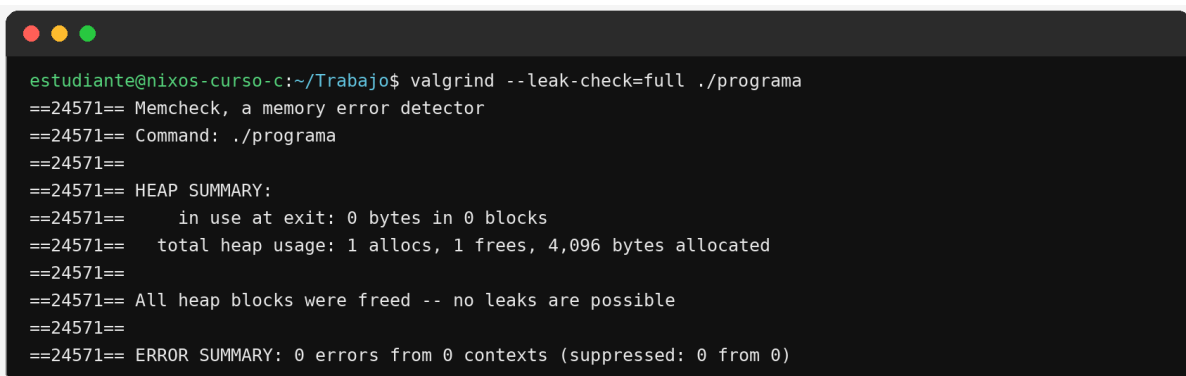
```
valgrind ./programa
```

Si el programa recibe argumentos, se escriben después del ejecutable:

```
valgrind ./programa archivo.txt 25
```

3 Una ejecución sin errores

Cuando no se detectan errores ni fugas, las últimas líneas se parecen a las de la siguiente imagen:



```
estudiante@nixos-curso-c:~/Trabajo$ valgrind --leak-check=full ./programa
==24571== Memcheck, a memory error detector
==24571== Command: ./programa
==24571==
==24571== HEAP SUMMARY:
==24571==    in use at exit: 0 bytes in 0 blocks
==24571==   total heap usage: 1 allocs, 1 frees, 4,096 bytes allocated
==24571==
==24571== All heap blocks were freed -- no leaks are possible
==24571==
==24571== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

Las dos frases más importantes son:

- All heap blocks were freed - no leaks are possible: toda la memoria reservada dinámicamente fue liberada.
- ERROR SUMMARY: 0 errors: Valgrind no encontró accesos inválidos ni otros errores durante esa ejecución.

Esto solo describe el camino que recorrió el programa con esa entrada. Un resultado limpio no garantiza que el programa esté libre de errores para todas las entradas posibles, puede ocurrir que con algunos testcases valgrind no te muestre errores pero con otros tests si.

4 Fugas de memoria

Una fuga ocurre cuando el programa reserva memoria con funciones como `malloc`, `calloc` o `realloc`, pero pierde la referencia a esa memoria sin liberarla con `free`.

El siguiente programa reserva un arreglo y termina sin liberarlo:

```

1 #include <stdlib.h>
2
3 int *crear_arreglo(void) {
4     int *datos = malloc(10 * sizeof(int));
5     return datos;
6 }
7
8 int main(void) {
9     int *datos = crear_arreglo();
10    if (datos == NULL) {
11        return 1;
12    }
13    datos[0] = 42;
14    return 0;
15 }
```

Al ejecutarlo con `-leak-check=full`, la salida puede verse como en la siguiente imagen:

```

estudiante@nixos-curso-c:~/Trabajo$ valgrind --leak-check=full ./fuga
==12345== Memcheck, a memory error detector
==12345== Command: ./fuga
==12345==
==12345== 40 bytes in 1 blocks are definitely lost in loss record 1 of 1
==12345==    at 0x4846828: malloc (vg_replace_malloc.c:431)
==12345==    by 0x10917A: crear_arreglo (fuga.c:5)
==12345==    by 0x1091C3: main (fuga.c:13)
==12345==
==12345== LEAK SUMMARY:
==12345==    definitely lost: 40 bytes in 1 blocks
==12345==    indirectly lost: 0 bytes in 0 blocks
==12345==    possibly lost: 0 bytes in 0 blocks
==12345==    still reachable: 0 bytes in 0 blocks
==12345==         suppressed: 0 bytes in 0 blocks
==12345==
==12345== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

Para leer este error:

1. La frase **definitely lost** indica memoria que el programa ya no puede recuperar ni liberar.
2. En el ejemplo aparece `crear_arreglo (fuga.c:5)`, este es el lugar donde se reservó la

memoria que presenta fugas.

En este caso, la corrección es liberar el arreglo cuando ya no se necesite:

```
free(datos);
```

4.1 Categorías de fugas

- **definitely lost**: fuga confirmada. Ya no queda ningún puntero válido hacia ese bloque. Como en el ejemplo anterior, se reservó memoria para un arreglo y después se pidió la referencia a ese arreglo sin liberarlo.
- **indirectly lost**: memoria que solo era accesible desde otro bloque que se perdió.
- **possibly lost**: Valgrind encontró un puntero que podría apuntar al interior de un bloque, pero no puede asegurarlo.
- **still reachable**: el programa terminó sin liberar el bloque, aunque todavía conservaba un puntero válido hacia él.

Como regla práctica, lo importante es que **definitely lost** e **indirectly lost** sean siempre cero.

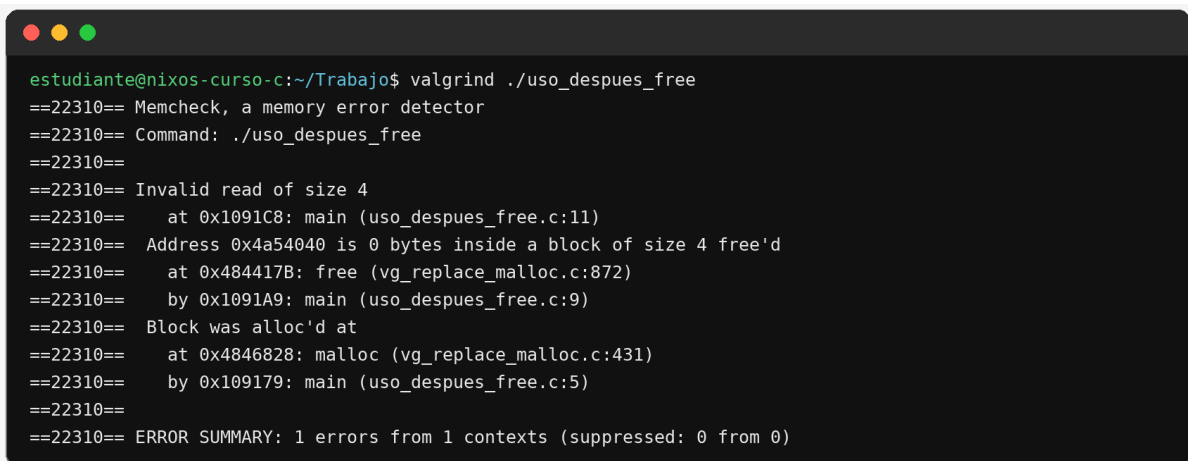
5 Accesos inválidos

Un `Invalid read` o `Invalid write` significa que el programa intentó leer o escribir en una zona de memoria que no podía usar correctamente.

Un caso común es acceder a un puntero después de llamar a `free`:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void) {
5     int *numero = malloc(sizeof(int));
6     if (numero == NULL) {
7         return 1;
8     }
9     *numero = 10;
10    free(numero);
11    printf("%d\n", *numero);
12    return 0;
13 }
```

La salida se parece a la siguiente imagen.



```
estudiante@nixos-curso-c:~/Trabajo$ valgrind ./uso_despues_free
==22310== Memcheck, a memory error detector
==22310== Command: ./uso_despues_free
==22310==
==22310== Invalid read of size 4
==22310==    at 0x1091C8: main (uso_despues_free.c:11)
==22310== Address 0x4a54040 is 0 bytes inside a block of size 4 free'd
==22310==    at 0x484417B: free (vg_replace_malloc.c:872)
==22310==   by 0x1091A9: main (uso_despues_free.c:9)
==22310== Block was alloc'd at
==22310==    at 0x4846828: malloc (vg_replace_malloc.c:431)
==22310==   by 0x109179: main (uso_despues_free.c:5)
==22310==
==22310== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

La salida entrega dos datos útiles:

1. `Invalid read of size 4`: el programa intentó leer cuatro bytes desde una dirección inválida.
2. `main (uso_despues_free.c:11)`: línea de tu programa donde ocurrió la lectura.

Este error puede ocurrir en muchos casos, por ejemplo: Definir un arreglo de tamaño `N` y escribir en una posición posterior a `N`; intentar trabajar con una variable que es `NULL` como si no lo fuera (intentar printearla, usarla como índice para un arreglo, etc); acceder a algo que ya fue liberado como vimos en el ejemplo anterior.

6 Valores no inicializados

También es posible usar una variable o una posición de memoria antes de asignarle un valor. Con `-track-origins=yes`, Valgrind intenta mostrar el origen del dato no inicializado.

```
estudiante@nixos-curso-c:~/Trabajo$ valgrind --track-origins=yes ./sin_inicializar
==31415== Memcheck, a memory error detector
==31415== Command: ./sin_inicializar
==31415==
==31415== Conditional jump or move depends on uninitialised value(s)
==31415==    at 0x1091B4: main (sin_inicializar.c:10)
==31415==    Uninitialised value was created by a heap allocation
==31415==    at 0x4846828: malloc (vg_replace_malloc.c:431)
==31415==    by 0x10917B: main (sin_inicializar.c:5)
==31415==
==31415== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
```

En este ejemplo:

- **Conditional jump or move depends on uninitialised value(s)** indica que una condición, como un `if` o un ciclo, dependió de un dato no inicializado.
- La primera traza muestra dónde se utilizó ese dato.

La solución suele ser asignar un valor inicial antes de leer la variable o inicializar correctamente todo el bloque reservado.

7 Cómo trabajar con la salida

Cuando Valgrind muestre muchos mensajes, sigue este orden:

1. Corrige el primer error que aparezca.
2. Compila nuevamente el programa.
3. Vuelve a ejecutar Valgrind.
4. Repite el proceso hasta que el resumen quede limpio.

Un error temprano puede provocar muchos mensajes posteriores. Por eso no siempre conviene intentar corregir toda la salida al mismo tiempo.

Durante las pruebas, usa distintas entradas para tu código. Valgrind solo puede revisar las instrucciones que realmente se ejecutan, por lo que si pruebas tu código con un testcase que no accede a todas las líneas de tu código, Valgrind no detectará errores en líneas que no fueron usadas.